CURLIX

Task Breakdown Document v1.0 — AI-Friendly Development Plan

This document breaks Curlix into small, manageable development tasks optimized for AI-assisted coding workflows. Instead of generating the entire project at once, each feature is isolated into focused implementation units to improve output quality, reduce token usage, and simplify debugging.

1. Why Task-Based Development

Large AI-generated projects often fail because: Context windows become too large Code consistency decreases Debugging becomes difficult AI loses architectural understanding Curlix is intentionally divided into isolated tasks so each feature can be: Built independently Tested independently Refactored safely Integrated gradually

2. Development Phases

Phase	Goal
Phase 1	Project setup and architecture
Phase 2	Database and backend core
Phase 3	Redirect engine and caching
Phase 4	Analytics and queues
Phase 5	Frontend dashboard
Phase 6	Security and anti-abuse
Phase 7	Deployment and optimization

Task 1 — Project Setup

• Initialize React + Vite frontend
• Initialize Express backend
• Configure environment variables
• Setup folder structure
• Configure GitHub repository

Task 2 — Supabase Database Setup

• Create Supabase project
• Design URLs table
• Design clicks table

• Configure indexes
• Test database connectivity

Task 3 — Backend Core Architecture

• Setup Express app structure
• Create controllers/services architecture
• Configure middleware system
• Setup centralized error handling

Task 4 — URL Shortening API

• Create POST /api/shorten endpoint
• Validate incoming URLs
• Generate nanoid short codes
• Save mappings into PostgreSQL

Task 5 — Redirect Engine

• Build GET /:code route
• Lookup Redis cache first
• Fallback to PostgreSQL on cache miss
• Return 302 redirects

Task 6 — Redis Integration

• Configure Upstash Redis
• Store short_code → long_url mappings
• Implement TTL expiration
• Add cache invalidation logic

Task 7 — Queue System

• Configure BullMQ
• Create analytics queue
• Push click events asynchronously
• Setup retry logic

Task 8 — Queue Workers

• Build worker process
• Save analytics into database

• Handle failed jobs
• Add worker logging

Task 9 — Analytics APIs

• Build GET /api/analytics/:code
• Aggregate click data
• Generate device statistics
• Create time-series analytics responses

Task 10 — Frontend Home Page

• Build URL input form
• Add loading states
• Display generated short URL
• Display QR code

Task 11 — Analytics Dashboard

• Create dashboard page
• Fetch analytics API data
• Render charts using Recharts
• Display click metrics

Task 12 — Owner Token System

• Generate secure owner tokens
• Hash tokens before DB storage
• Store tokens in localStorage
• Validate tokens on delete requests

Task 13 — CAPTCHA Integration

• Configure Cloudflare Turnstile
• Verify CAPTCHA tokens backend-side
• Block automated submissions

Task 14 — Rate Limiting

• Setup Redis-backed rate limiting
• Protect shortening endpoint
• Protect redirect endpoint

• Return 429 responses

Task 15 — Security Middleware

• Configure Helmet.js
• Add CORS restrictions
• Add request sanitization
• Prevent reserved alias usage

Task 16 — Deployment

• Deploy frontend to Vercel
• Deploy backend to Render
• Configure environment variables
• Connect Supabase and Redis

Task 17 — Render Cold Start Fix

• Setup UptimeRobot monitoring
• Ping backend every 14 minutes
• Prevent Render sleeping

Task 18 — Final Optimization

• Improve API performance
• Optimize cache hit ratio
• Test scalability
• Fix production bugs

3. AI Usage Strategy

Recommended AI workflow: Ask AI to solve only one task at a time Never generate the full application in one prompt Complete backend before frontend polishing Test each task before moving forward Keep prompts architecture-focused Benefits: Higher code quality Cleaner architecture consistency Fewer hallucinations Better debugging experience

Document prepared for Curlix Engineering — Task Breakdown v1.0